FACULTAD DE INGENIERÍA Y CIENCIAS

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe Tarea 2: Torneo de Komikan Concurrente

Sistema Operativo

Sección : 2

Profesor: Jonathan Frez

Alumnos: Jasson Valverde , Carlos Araya

1. Introducción

El juego mapuche tradicional Komikan consiste en un puma contra 12 perros en un tablero de 5x5 que intersecciona a un trono. Para esta tarea lo convierte en un torneo multijugador en **tiempo real y sin turnos**, donde las fichas intentan moverse de forma simultánea con retardos de forma aleatoria (50 a 300 ms por defecto). El objetivo es la implementación de la infraestructura completa, como un servidor de emparejamiento (`matchmaker`), hasta `M` mesas presentes y un juez por mesa que interviene la partida, pero utilizando primitivas POSIX (como `pthread_mutex, pthread_cond, sem_t`). Ademas se implementa un archivo config.txt. El archivo de configuración está definido como cada valor , está en parámetros como semilla, los retardos mínimo y máximo, la disposición inicial de las fichas y el límite de movimiento (aunque este es opcional), el porcentaje de abandono y el tiempo máximo de llegada.

2. Diseño del Esquema de Sincronización

- **Arquitectura:** El programa al momento de ejecutar `./torneo N M [config.txt]` procede a crear un hilo (`matchmaker`), `N` hilos de clientes (uno por cada jugador) y por cada partida activa, un hilo Juez, un hilo para el puma y otros 12 hijos para los perros (corresponden a 14 hilos por mesa) ,ademas del `matchmaker` y los N clientes. Para el tablero se modela con 28 intersecciones, una grilla de 5x5 (casillas 0-24) , trono (casilla 25) y los dos nodos del techo (casillas 26, 27), con tablas de adyacencia (`ady`) y de los saltos del puma (definido como `saltospum`, en los tramos a-b-c en línea recta) que son calculadas una vez antes de lanzar los hilos y luego de solo lectura.

- **Exclusión mutua en los movimientos:** En cada casilla de la mesa tiene su propio mutex (`pthread_mutex_t`) con tal de garantizar en que nunca hayan más de una ficha por casilla y su estado solamente cambia dentro se su sección crítica. Luego una ficha se duerme con un retardo de forma aleatoria (`[MIN , MAX]`) en ms, además elige un destino (`ady`) y toman a los mutex de origen y destino de manera ascendente de índice. Mencionado anteriormente, dentro de la sección crítica verifica de que el destino siga vacia (estado vacia, no NULL) y que la ficha siga vigente (estado `Perro` y su `idpie` propio), en caso de que sea así se mueven y actualiza el tablero, de lo contrario, incrementa el `conflic` y reintentar nuevamente en el siguiente ciclo. Además como la corroboración y la escritura ocurren bajo el mismo lock, solo una ficha obtiene una casilla disputada. El salto del puma realiza un bloqueo en tres casillas (en origen - perro saltado - destino), también se ordenan y se verifica que las casillas Perros + Vacío antes de realizar la captura. Por ende los mensajes de cada movimiento se imprimen después de soltar los locks.

- **Juez de mesa:** El `matchmaker` procede a preparar la mesa con `inicio_mesa` antes de crear al Juez. El Juez procede a lanzar 13 fichas, observa el estado y determina el resultado. Por cada 20 ms toma las 28 casillas (`bloquear()`, de manera ascendente) para obtener una copia de **snapshot** de manera consistente, luego se evalúa la victoria (como en 6 o más capturas, puma sin movimiento ni saltos, o por el límite de movimiento) y las sueltan en orden inverso. Luego imprime el tablero de cada 2 s y los eventos que son claves para cada movimiento. Al finalizar, el estado de la partida queda como inactiva , después se esperan con `pthread_join` los 13 hilos, imprimen el tablero final, se destruye los 28 mutex, avisan a ambos jugadores y se libera la mesa. Para la validez de cada movimiento (ya sea adyacencia o captura en línea recta) se garantiza por construcción , ya que las fichas solo eligen los destinos de las tablas precalculadas y escriben bajo lock.

- **Resiliencia:** Cuando un jugador marca como fallido (con porcentaje configurable) se espera entre 300 a 1500 ms sobre su semáforo `fin`, en caso de que la partida siga vigente con el mismo número de partida, activa y sin abandono (con parámetro fijo `aband` y bajo `estmax`). Y el juez lo detecta, se declara ganador al jugador oponente, luego se detiene y espera a las fichas y destruye las primitivas de la mesa. Con el comando `Ctrl + C` , el manejador de `SIGINT` solo escribe el seguro en señales , preserva `errno` (con el servidor `servidor_act = 0`) . Los clientes proceden a dejar de encolarse, luego el `matchmaker` procede a cerrar la cola y despertar con `mesa = NULL` a quienes no tienen rival, luego los jueces terminan sus partidas con el resultado como "interrumpido", y el `main` espera con `pthread_join` a todos los hilos, se destruyen los semáforos y mutex , y liberan la memoria dinámica, para que el programa termine con código 130

3. Análisis de Riesgos de Concurrencia

- **Deadlock:**Podría producirse si dos perros se cruzan (A a B y/o B a A), si un salto del puma comparte casillas con un perro, o si el juez toma las 28 casillas mientras que las fichas toman 2 o 3. Para prevenirlo, con un **orden total de adquisición** por los índices de las casillas, aplicado en perros, pumas (tres casillas ordenadas e intercambiables) y un juez, con la liberación del orden inverso, ya sea sin ciclos de espera no hay más deadlock. Además, que ningún hilo pide un mutex globales (como `imprime , stat, mtx_*`) que retienen las casillas y el único contenido entre mutex globales (como `estmax` y luego lo imprime) va en un solo sentido. De manera cronológica se evita el bloqueo permanente de los jugadores, ya que todo camino de salida retorna el semáforo correspondiente (el `matchmaker` procede a despertar con `mesa = NULL` a quienes descarta y el juez pública el fin de ambos jugadores), y las esperas del `matchmaker` utilizan timeouts de 100 ms para revisar la bandera (flag) del comando Ctrl + C.

- **Livelock:** Podría producirse si las fichas que se chocan , hace el reintento de sincronizarse nuevamente o a nivel de la partida, si las fichas se mueven de manera eterna sin haber un solo ganador. Los retardos son de forma aleatoria con su semilla propia por hilo (`rand_r` que es un derivado de la semilla, la partida y el id) y sin spin, ya que cada reintento espera por un nuevo retardo entre 50 a 300 ms. A nivel de la partida, el límite de movimientos (`lim_mov`) fuerza un empate y procede a terminar la partida

- **Starvation:** Los mutex POSIX no son de algoritmo FIFO, ya que una ficha (o el juez) podrían perder repetidamente una casilla muy cuestionada, o un jugador espera la mesa de manera indefinida. Luego se mitiga con la secciones críticas de microsegundos entre 50 - 300 ms entre intentos y reintentos del juez cada 20 ms, la cola de los jugadores con algoritmo FIFO y solo el matchmaker hace la espera en `ctr_mesas`, por lo que ningún par es adelantado. El riesgo residual se dice que no hay una garantía formal de equidad entre las fichas individuales.

- **Carreras de Control:** Al momento de reutilizar una mesa mientras que termina su juez anterior, un jugador fallido que afecta a la próxima partida y los despertares perdidos de la variable de condición se resuelven de esta forma, el matchmaker hace un `join` del juez anterior antes de reiniciar la mesa, el número de partida (`part`) se valida antes de marcar como abandono, la espera de la cola utiliza un while y un timeout, y por último una ficha capturada se detiene al ver que la casilla ya no le pertenece.

4. Evaluación

- **Locks por casilla en lugar de un lock por mesa**: Se podría utilizar un lock global ,sería más simple (sin orden de adquisición ni locks para saltos), pero se serializaría todo los movimientos de la mesa. Por ende se eligió la granularidad fina porque nos permite que cada movimiento en las casillas se progresen en paralelo, a cambio de complejidad moderada, ya que una regla de orden único, que además permite la facilidad de razonar la ausencia de deadlock. El juez es el único que puede bloquear todo el tablero y lo hace de manera breve para obtener una foto consistente. Además los contadores y las banderas de Ctrl + C son atómicos ya que no permite ampliar las secciones críticas.

- **Juez por sondeo en lugar de notificación:** Al detectar el fin de una variable de condición, se exigirá que cada ficha avisara tras cada movimiento, en cambio el sondeo es más simple y su latencia (intervalo nominal de sondeo de 20 ms) ya que es despreciable frente a movimientos entre 50 - 300 ms.

- **Rendimiento Medido:** En una partida (con semilla 42 , retardo 50 - 300 ms) se estima una aproximación de 126 movimientos y 94 intentos sobre las casillas que están ocupadas, tiene una duración de 4,7 s. Con 8 jugadores (equivale a 4 partidas) , con M = 2 tiene duración 7,4 s y M = 4 tiene una duracíon de 5,0 s y el máximo de mesas de manera simultánea nunca supera M mesas. Con la misma semilla , los totales coinciden entre ambas corridas (351 movimientos y 398 conflictos). Esto produjo un cuello de botella que el retardo artificial de las fichas, no los locks. La contención apreciada (un conflicto por cada 1 - 1,5 movimiento en tablero poblado) se resuelve de tal forma de que no hay espera largas porque el conflicto sólo implica leer el estado y reintentar nuevamente.

5. Conclusión

El sistema del Torneo de Komikan cumple lo pedido por el enunciado, un matchmaker empareja a los N jugadores, un semáforo contador que limita las partidas a M mesas y un juez arbitra cada una en tiempo real. Un mutex por casilla y un único orden de adquisición que evitan por diseño las carreras y el deadlock sobre el tablero, y los retardos aleatorios junto al límite de movimientos que previenen el livelock y aseguran que toda partida termine. El abandono y Ctrl+C se manejan con cierre ordenado y sin fugas.